CineMacondo
Cinema Management System

1st Alejandro Nuñez Barrera
Universidad Distrital Francisco Jose de Caldas
Bogota, Colombia
anuzeb@udistrital.edu.co

2rd Miguel Angel Sanabria Pineda
Universidad Distrital Francisco Jose de Caldas
Bogota, Colombia
masanabriap@udistrital.com

***Abstract*—The main problem that our application is aiming to resolve is the tedious process of buying tickets for a movie physically, because this could be very slow and annoying tasks. The solution that we propose to this problem is an application that allows the users to buy tickets online in an easiest and fastest way that the process would traditionally be, even having access to exclusive memberships through the application. The most relevant result of all the work is the final application that allows the user to manage their memberships, buy various tickets through a cart system and make the payment with credit card or PayPal.**

***Index Terms*—Cinema, Application, Design Patterns, Ticket, Purchase**

I. Introduction

The principal focus of the application is to solve a problem that many people have, which is the annoying process that buying a ticket for a movie physically can be. This issue becomes even worse when there is a premiere of a movie, and the people flow is much bigger than usual, so the lines for buying the tickets are longer and more time-consuming than they are on a normal day. The inconvenience of waiting in long queues, dealing with limited ticket availability, and the possibility of not getting a seat in a preferred location make the traditional ticket purchasing process frustrating for many users. Additionally, the manual process at ticket counters can lead to human errors, causing incorrect bookings or delays in processing transactions, further contributing to user dissatisfaction.

There have been previous solutions for this problem, each one of them made for a theater in particular. For example, the apps and websites of Cinemark and Cinecolombia, these two platforms let the user have their own profile that allows them to look for the movies that are currently on the billboard in a preferred location, manage their purchases, and in some theaters, manage exclusive memberships and offers. With these features, they also solve the problem of the long, time-consuming, and annoying process of doing it physically. However, these platforms are limited to specific theaters, meaning that users who visit different cinema chains may need to use multiple applications to buy tickets, making the experience less convenient. Additionally, some platforms have issues such as slow loading times, complex interfaces, and limited payment options, which can discourage users from using them frequently.

The techniques that our app uses for being optimized are the different design patterns used in the back-end for making a faster and easy-to-use application. The design patterns [1] are developer tools that are used to provide a solution to common problems encountered throughout the development of an application. These can help developers to make a more flexible, sustainable, and better application overall. Some of the design patterns implemented include the Repository Pattern, which ensures better separation of concerns and improves maintainability, and the Factory Pattern, which simplifies object creation and enhances code re-usability. Along with that, we used three tools to make the development easier. These are: Docker [2], a platform for developing, shipping, and running applications, helping us when performing tests by reducing the time it takes to write the code and test it, making it easier for us; Spring Boot [3], a framework for Java applications that also helps at the time of configuring the application and running it in a fast way, reducing boilerplate code and allowing a more efficient integration with the database and external APIs; and Postman [4], a platform that we used mainly for testing every process in the application, making it very easy to set up the parameters for performing a test and executing it.

Overall, by combining the efficiency of design patterns, development tools, and automation practices, the application ensures a seamless, user-friendly experience that eliminates the inconvenience of buying movie tickets physically and provides a modern solution to a long-standing problem.

II. Method and Materials

The initial design of the application will be a command-line based program that would allow the user to make use of each one of the features with only typing the required data on the console. It will be developed between two programming languages, Java and Python. Most of the application will be developed in Python, and that is due to the many benefits that this language has for developing an application like this. It allows a faster development thanks to being less strict in some aspects, it has various libraries and frameworks that allow making a better program in a faster

Identify applicable funding agency here. If none, delete this.

way, it is easier to automate tasks in it, allows portability and flexibility, and has a very strong community support that helps solve any problem that could appear during the development. Python also provides a variety of built-in modules and third-party packages that simplify tasks like data processing, API consumption, and file handling, reducing the amount of manual coding required. Additionally, the simplicity of Python's syntax allows for better readability and maintainability of the code, making it easier for future developers to understand and modify the application when needed.

While Java is only used for the payment process, it does not mean that it does not have any benefits, because thanks to the frameworks and dependencies, it allows difficult tasks to become simpler. For example, when connecting to the database, we do not have to manually code the query if they are simple inserts, selects, or so, because there are dependencies that auto-generate the query. Also, at the time of implementing payment methods like the PayPal one, it has its own dependency that allows performing tests without the need for an actual card. Another great advantage of Java in this case is its robustness and security features, which are essential when dealing with sensitive payment information. Since Java is widely used in enterprise applications, it ensures a stable and scalable architecture that can handle multiple concurrent transactions without performance issues. Java's strong type system and built-in exception handling also contribute to creating a reliable payment system, preventing unexpected errors that could affect the user experience.

The application also implements a database made in PostgreSQL where all of the data is saved, such as users, orders made, movies on the billboard, etc. The benefits of having a database are that it helps to maintain the integrity of the data as well as its consistency, preventing data issues like users with the same ID, email, or other unique attributes. It also maintains the data organized within tables, allows multiple users to enter at the same time, and most importantly, provides a backup of the data in case any problems happen and it gets corrupted. Additionally, PostgreSQL offers advanced features like indexing, foreign keys, and constraints, ensuring that data retrieval is efficient and relationships between tables are properly maintained. It also supports complex queries and transactions, which allows handling large datasets effectively. The use of PostgreSQL also ensures better security measures, as it includes authentication mechanisms and encryption support to protect sensitive information. Moreover, its ability to handle high volumes of data with stability and reliability makes it a great choice for an application that requires real-time access to information, such as tracking orders and managing user accounts.

To further enhance the application's usability and performance, logging and error-handling mechanisms will be implemented to track issues and ensure a smooth user experience. Overall, the combination of Python, Java, and PostgreSQL ensures an efficient, scalable, and maintainable application that can be expanded as needed.

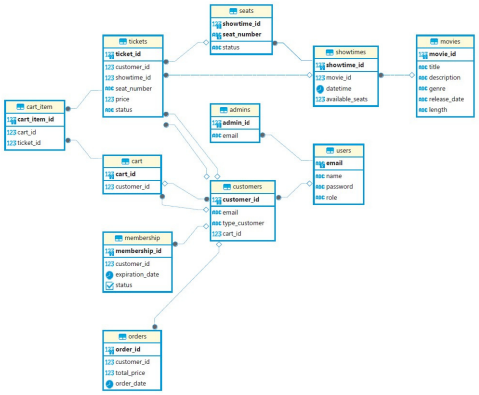

Image of the Entity-Relation diagram of the database

III. Results

To test the final result of the application, as mentioned before, we utilized tools such as Spring Boot to run the Java application on a designated port, while Postman was used to test various requests and validate the functionality of the Java part of the system. In the case of the Python component, we leveraged Docker to run the application on the desired port, ensuring a controlled environment for execution, and once again relied on Postman to test different requests and verify that most of the application's features performed as expected. The results of these tests were satisfactory, as the GET requests consistently returned the expected values, confirming that data retrieval operations worked correctly, while the POST requests successfully performed their intended actions, such as creating new users and storing relevant information.

Additionally, further tests ensured that both the Java and Python services communicated effectively, demonstrating system stability, reliability, and consistency across different functionalities. This testing approach not only validated the correctness of individual components but also confirmed seamless integration between the services, reinforcing the overall robustness of the application.

IV. Conclusions

In conclusion, we developed an application based on Java and Python that incorporates the use of different design patterns to help create a more optimized and efficient system, solving common problems that could arise during development. These design patterns not only improve the flexibility and maintainability of the code but also contribute to making the application more scalable and adaptable to future enhancements.

With the help of Docker, we ensured a consistent development and testing environment, reducing compatibility issues across different systems. Spring Boot played a crucial role in simplifying the Java-based back-end, making configurations easier and improving overall performance, especially in handling the payment process. Postman was used extensively for testing, allowing us to validate every request, response, and

process within the application, ensuring that all functionalities work as expected before deployment. These tools collectively contributed to making the development process smoother and more efficient, reducing the chances of errors and improving overall reliability.

Additionally, we prioritized the security and integrity of data by using a relational database based on PostgreSQL, ensuring that all stored information is structured, consistent, and protected from unauthorized access. This database design makes the process of storing, retrieving, and managing data more efficient, while also supporting concurrent user access without performance issues.

Finally, we successfully achieved our main goal: solving the problem of purchasing movie tickets in a fast and convenient way. The application allows users to buy their tickets online, eliminating the long and time-consuming process of purchasing them physically. With an optimized back-end, and secure data handling, we created a modern solution that improves the movie-going experience for users, making it more accessible.

BIBLIOGRAPHY

Please number citations consecutively within brackets [1]. The sentence punctuation follows the bracket [2]. Refer simply to the reference number, as in [3]—do not use “Ref. [3]” or “reference [3]” except at the beginning of a sentence: “Reference [3] was the first . . .”

Number footnotes separately in superscripts. Place the actual footnote at the bottom of the column in which it was cited. Do not put footnotes in the abstract or reference list. Use letters for table footnotes.

Unless there are six authors or more give all authors’ names; do not use “et al.”. Papers that have not been published, even if they have been submitted for publication, should be cited as “unpublished” . Papers that have been accepted for publication should be cited as “in press” . Capitalize only the first word in a paper title, except for proper nouns and element symbols.

For papers published in translation journals, please give the English citation first, followed by the original foreign-language citation .

REFERENCES

[1] GeeksforGeeks, Software Design Patterns tutorial, GeeksforGeeks, January 2nd 2025. https://www.geeksforgeeks.org/software-design-patterns/.

[2] “What is Docker?”, Docker Documentation, 10th Septemeber 2024. https://docs.docker.com/get-started/docker-overview/

[3] Ibm, ¿Qué es Java Spring Boot? — IBM, IBM, 17th July 2023. https://www.ibm.com/mx-es/topics/java-spring-boot

[4] What is Postman? Postman API Platform. https://www.postman.com/product/what-is-postman/